Module 01 · Week 3

Citizen-Facing Service Prototype.

Hackathon handbook.

building, testing and presenting a GOV.UK service with AI.



This handbook is your reference for the Week 3 hackathon, your first assessed portfolio piece. It explains the ideas behind the brief: how to plan a small service with your team and with AI, the GOV.UK patterns the brief asks for, what accessibility means for a public service and how to test it, how to prove your eligibility rules with unit tests, and how to present and reflect on your work. Use it alongside the brief, the lab README and the hackathon pack. Each chapter ends with questions to check your understanding and places to read further.

Table of contents

1. How the Hackathon Works **4**
- What you will be able to do 4
- The Minimum Viable Submission 4
- Checkpoints and assessment 5
- Working as a team with AI 5

2. Planning a Service with AI **6**
- The component tree 6
- One state object 6
- Eligibility rules are your decision 6
- Ground Claude in the design system first 7
- Recording decisions 7

3. GOV.UK Patterns in the Brief **8**
- Start page 8
- Question pages: one thing per page 8
- Check your answers 8
- Error summary and error messages 9
- Result and confirmation 9
- Service Standard signals 9
- Teaching scaffold or production library? 9

4. Accessibility for Public Services **11**
- The law and the standard 11
- The criteria this brief tests most 11
- A manual walkthrough 12
- Automated checks 12

5. Testing Your Eligibility Rules **14**
- Why a pure function is easy to test 14
- Setting up Vitest 14
- Choosing your five tests 14
- Test first, and who decides "correct" 15
- End-to-end tests (stretch) 15

6. Working with AI as a Team **16**
- The AI log 16
- Where AI-generated code usually needs correcting 16
- Ways of working that teams found useful 17
- Models and usage limits 17

7. Presenting and Reflecting **18**
- The presentation 18
- Peer feedback 18
- Reflection 18

References and Further Reading **20**

Appendices **22**
- A. Setup and command cheat sheet 22
- B. Glossary 22

C. Templates 23
D. Accessibility walkthrough checklist 24
E. Suggested answers to Check your understanding 25

1. How the Hackathon Works

Week 3 is a hackathon: a short, focused team build against a brief. Your team designs, builds, tests and presents a digital service for the Green Home Grant, a fictional government scheme that funds home insulation and heat pumps. Today, citizens phone a call centre and wait an average of 35 minutes to find out whether they qualify. Your service replaces that call.

The week brings together everything from the first two weeks: working with Claude Code, React components and state, the GOV.UK design patterns, accessibility and testing. It is also your first assessed portfolio piece, something you can show in an interview or a performance conversation as evidence of what you can build.

What you will be able to do

- Plan a small service as a team, with AI, before building it.
- Build a GOV.UK start page, a one-thing-per-page question flow, a check your answers page and a result page.
- Hold form answers in one place so they survive navigation between pages.
- Write eligibility rules as a pure function and prove them with unit tests.
- Test a service against WCAG 2.2 AA, and explain why that is a legal requirement for public services.
- Record AI-assisted work in an AI log that shows your own judgement, not just that AI was used.
- Demo a working service and give structured feedback on someone else's against a rubric.

The Minimum Viable Submission

The brief lists ten requirements. Six of them form the **Minimum Viable Submission (MVS)**: the floor every team must ship before any stretch goal or polish. If you ship these six cleanly, you have met the bar.

#	MVS item	What it means
1	Multi-page question flow	A start page and at least five question pages, one question per page
2	Check your answers	A summary of every answer, each with a Change link
3	Confirmation page	A clear outcome: eligible, not eligible or partially eligible
4	GOV.UK components	Typography, spacing and headings from the provided CSS variables
5	WCAG 2.2 AA	Labelled inputs, keyboard use, 4.5:1 contrast, visible focus
6	AI usage log	`AI_LOG.md` with at least three entries, four fields each

The other four requirements (responsive layout at 320px, GOV.UK error pattern, at least five unit tests, and GOV.UK typography in full) are still assessed. Build outward from the floor.

Checkpoints and assessment

Facilitators check in at a few fixed points: a **first green path** (one route from start page to result page that renders, however rough), a **first checkpoint demo** (start to result with Continue, Back and saved answers), a **mid-point check** (check your answers and one validation error working), the **presentation**, and the **final submission**. Only the final submission is pass or fail. The checkpoints tell you, and the facilitators, where help is needed.

Your work is judged against a six-row rubric: functionality, code quality, AI effectiveness, accessibility, GOV.UK visual compliance, and Service Standard conformance. Treat it as your client's definition of done. You can check yourself against its "Excellent" column at any time.

Working as a team with AI

One way to think about an AI-augmented team: instead of four programmers, you have **four managers and one very hard-working programmer**. Each of you directs the AI, reviews its output and decides what ships. The AI does the typing; you carry the judgement. Your skill is in the clarity of what you ask for and the sharpness of what you keep.

Teams that do well in a hackathon plan first, agree objectives and check-points, share the work deliberately (one owner per page, or pairs where one person drives and one reviews), and ship the floor before anything else. An hour spent agreeing how you will share code is an hour well spent.

Key idea. Claude is your intern developer. Check every change it says it has made before you give it the next prompt.

Check your understanding

1. Name the six MVS items.
2. What is the only checkpoint that is pass or fail?
3. In the "four managers and one programmer" framing, what is each team member's job?

2. Planning a Service with AI

The brief is intentionally open. It fixes the end state (a working service that meets the acceptance criteria) but not the component structure, the prompts you use or the order you build in. Those are your decisions, and planning is where you make them.

The component tree

Start with a sketch of the pages and how a user moves between them: a start page, five question pages, a check your answers page and a result page, plus the header, phase banner, footer and accessibility statement that appear around them. The starter scaffold already has a route for each page, so the sketch is mostly about who builds what, and in which order.

One state object

The single most common failure in this build is losing answers between pages. If each question page keeps its own answer with `useState`, the answer disappears the moment the page unmounts: the check your answers page shows blanks, and pressing Back loses what the user already entered.

The fix is React's **lifting state up**: hold every answer in one object in the component that contains all the pages (`App`), and pass each page the value it needs and a function to update it.

```
const [formData, setFormData] = useState({
  propertyType: "",
  ownership: "",
  incomeBand: "",
  insulation: "",
  heating: ""
});
```

Eligibility rules are your decision

The brief tells you what to ask, but not how the answers decide eligibility. Your team decides the rules (for example, which income bands qualify, whether renters can apply, what counts as already insulated) and writes them down. That is a real part of the work: in a government service, the rules would come from policy, and the team would record how each was interpreted.

Keep the rules out of the pages. Write them as one **pure function** in its own file: it takes the state object and returns an outcome and a reason, and does nothing else. Pages collect answers; the function decides. Chapter 5 explains why that makes it testable.

Ground Claude in the design system first

Before your first component prompt, add a `CLAUDE.md` to your project. Claude Code reads it at the start of every session, so prompts like "add a start page" produce GOV.UK-patterned markup without you repeating the constraints each time. Keep it short and concrete:

```
# Project context
This is a GOV.UK service. All UI must follow the GOV.UK Design System.
- Use the govuk-* classes defined in src/App.css. No custom colours.
- One question per page. Continue button, Back link.
- Summary list is a <dl> with govuk-summary-list classes.
- Error summary: role="alert", heading "There is a problem",
  links to each field in error.
- Every input has a <label>. Do not add features we did not ask for.
```

Plan mode (`Shift+Tab`) is a good way to ask Claude for an architecture: it can read your files and propose a plan, but cannot change anything. Review the plan as a team, change what you disagree with, and record it as your first AI log entry.

Recording decisions

Some teams go further than the AI log and keep **decision records**: short, dated notes of a decision, the options considered and why one was chosen. An Architectural Decision Record (ADR) is the standard form. Others keep a **RAID log** (risks, actions, issues, decisions). Neither is required, but both make your reasoning visible, which is what an assessment panel looks for.

Check your understanding

1. Why do answers disappear when each page holds its own state, and what is the fix?
2. Why should the eligibility rules live in their own function rather than in the pages?
3. What is `CLAUDE.md` for, and when does Claude Code read it?

3. GOV.UK Patterns in the Brief

GOV.UK services look and behave consistently because they use the same patterns. The brief grades four of them. Each exists for a reason that comes from research with real users, and knowing the reason helps you spot when AI-generated code gets it wrong.

Start page

A service starts with a page that says what it does, gives the information most users need before they begin, and has a prominent green **Start now** button. On GOV.UK itself, start pages are content pages that link into the service; in this prototype, your start page plays that role.

Question pages: one thing per page

Each question lives on its own page with a single input and a **Continue** button. It reduces the load on the user, makes validation unambiguous, and means any user can answer "what is this page asking?" in one sentence. The Design System's guidance for question pages also says:

- Every page has a **Back** link near the top: users often distrust the browser's back button while filling in a form.
- The page heading is the question, and it can also be the label (or, for radio buttons, the legend), so screen reader users only hear it once.
- The button says "Continue", not "Next".
- Mark optional fields as optional; do not mark required fields with asterisks.

Check your answers

Before a service does anything that cannot be undone, it shows the user everything they entered on one page, with a **Change** link beside each answer. Two details matter:

- The list is a **summary list**, built with a description list (`<dl>`), not a `<table>`. A table implies relationships between rows that key-value pairs do not have, and screen readers announce it wrongly. AI assistants often generate a table here.
- Each Change link has visually hidden text, so a screen reader user hears "Change property type" rather than five identical "Change" links. After a change, the user returns to the check your answers page, not to the next question.

Error summary and error messages

When a user submits a page with a problem, GOV.UK services show both an **error summary** at the top of the page and an **error message** beside the field.

Part	What it needs
Error summary	Placed at the top of the main content; heading "There is a problem"; `role="alert"`; a link from each error to the field in error; focus moves to it when the page loads
Error message	Beside the field, worded exactly as in the summary
Page title	Starts with "Error: " so screen readers announce the problem immediately

Write specific messages. "This field is required" tells the user nothing; "Select the type of property you live in" tells them what to do. Use `role="alert"`, not `alertdialog`: an alert dialog traps focus and demands acknowledgement, which stops a screen reader user moving freely. An error summary that is only a styled box looks right and fails the pattern.

Result and confirmation

The result page gives a clear outcome (eligible, not eligible or partially eligible), the reason, and what to do next. GOV.UK confirmation pages use a prominent panel for the outcome, followed by plain-English next steps.

Service Standard signals

Real GOV.UK services show a few things that tell users they are in a proper government service. The rubric's last row asks for four:

- a **phase banner** under the header, saying the service is in alpha or beta, with a feedback link;
- a **footer** with a link to the accessibility statement;
- an **accessibility statement**, which public sector services are legally required to publish (see Chapter 4);
- a **cookie notice**.

Teaching scaffold or production library?

The starter uses plain CSS variables and simple React components so you learn the patterns from first principles. Production services use **govuk-frontend** (the official package) or **govuk-react** (a community binding), which ship tested components that handle accessibility, responsive behaviour and print styles for you. If you mix the two, expect conflicts: your own CSS and govuk-frontend's styles will fight, and Claude may offer to use one or the other rather than combine them. Decide as a team which you are using, and say so in `CLAUDE.md`.

Check your understanding

1. Why is the check your answers summary a `<dl>` and not a `<table>`?
2. What three things does a GOV.UK error summary need beyond its visual style?
3. Rewrite "This field is required" for the heating question.
4. Name the four Service Standard signals in the rubric.

4. Accessibility for Public Services

A citizen-facing service is used by people across the full range of devices, connection speeds and access needs, including people who use a screen reader, only a keyboard, or a phone held in one hand on a bus. A checker that a disabled citizen cannot complete has failed the people the grant exists to reach. That is why accessibility in this brief is a requirement, not a finishing touch.

The law and the standard

The **Public Sector Bodies (Websites and Mobile Applications) (No. 2) Accessibility Regulations 2018** require public sector websites and apps to meet **WCAG 2.2 at level AA**, and to publish an accessibility statement that explains how accessible the service is and how to report problems. GDS monitors a sample of public sector sites each year, and the Equality and Human Rights Commission enforces the requirements. GOV.UK publishes a sample statement you can adapt.

The criteria this brief tests most

WCAG 2.2 criterion	Level	What it means for your service
1.3.1 Info and relationships	A	Structure is in the code: real headings, labels tied to inputs, a `<dl>` for the summary
1.4.3 Contrast (minimum)	AA	Normal text has at least 4.5:1 contrast with its background
1.4.10 Reflow	AA	Content works at 320 CSS pixels wide without scrolling sideways
2.1.1 Keyboard	A	Everything can be done with a keyboard alone
2.4.7 Focus visible	AA	You can always see which element has keyboard focus
2.4.11 Focus not obscured (minimum)	AA	New in 2.2: the focused element is not hidden behind other content
2.5.8 Target size (minimum)	AA	New in 2.2: controls are at least 24 by 24 CSS pixels, with some exceptions
3.3.1 Error identification	A	Errors are described in text, not just shown with colour
3.3.2 Labels or instructions	A	Every input has a label
3.3.7 Redundant entry	A	New in 2.2: users are not asked for the same information twice

WCAG 2.2 became a W3C Recommendation in October 2023. It added nine success criteria and removed one (4.1.1 Parsing). Some of the new ones, such as focus appearance, are level AAA and not required here. If you use GOV.UK Frontend components correctly, several of the 2.2 criteria are covered for you; with the teaching scaffold, you have to check them yourself.

A manual walkthrough

Automated tools find some problems, such as missing labels or invalid attributes, but many can only be found by a person. Do these checks yourself, in this order:

1. **Keyboard.** Put the mouse aside. Starting from the address bar, press `Tab` through every interactive element. Every element should get visible focus, in a sensible order, and you should never get stuck. Choose options with `Space` and buttons with `Enter`. Complete the whole journey.
2. **Labels.** Clicking a label should select its option. Every input needs one.
3. **Contrast.** In browser DevTools, inspect some text and open the colour picker: most browsers show the contrast ratio. It must be at least 4.5:1.
4. **Reflow.** Turn on DevTools' device toolbar and set the width to 320 pixels, or zoom to 400%. Go through every page. Nothing should scroll sideways, overlap or be cut off.
5. **Screen reader spot-check.** Use VoiceOver on a Mac, NVDA on Windows or Orca on Linux, or the Accessibility pane in DevTools. Go to one form field: you should hear its label and, if there is an error, the error message.

Automated checks

Automated scanners are a useful second line. Pa11y runs from the terminal against your running app:

```
npx pa11y http://localhost:5002 --standard WCAG2AA
```

Run it in the lab machine's terminal, where `localhost` is correct. For checks that run every time, `@axe-core/playwright` adds accessibility scans to Playwright end-to-end tests. Neither replaces the manual walkthrough.

Key idea. Accessibility is tested, not assumed. Claude often leaves out ARIA attributes and labels unless you ask, and code that looks right can still fail the keyboard test.

Check your understanding

1. Which regulations require UK public sector services to meet WCAG 2.2 AA, and what must they publish?
2. What contrast ratio does normal text need, and how do you check it?
3. Why is a manual keyboard test still needed if an automated scanner reports no errors?

5. Testing Your Eligibility Rules

The brief asks for at least five unit tests of your eligibility logic. This is where writing the rules as a pure function pays off.

Why a pure function is easy to test

A **pure function** only uses what it is given and only returns a result: the same answers in always give the same result out. It does not read from the page, change the page or depend on anything else. So a test can call it directly with a state object and check what comes back, without starting the app or rendering anything.

Setting up Vitest

The starter does not include a test runner. **Vitest** is designed for Vite projects. The latest Vitest needs a newer Vite than the starter uses, so install version 1, which matches the starter:

```
npm install -D vitest@1
```

Add a test script to `package.json`, then put tests in files whose names contain `.test.`, next to the code they test:

```
"scripts": {
  "dev": "vite",
  "test": "vitest run"
}
```

```
import { describe, it, expect } from 'vitest';
import { checkEligibility } from './eligibility';

describe('checkEligibility', () => {
  it('says council tenants are not eligible', () => {
    const result = checkEligibility({ ownership: 'rent-council' });
    expect(result.status).toBe('not-eligible');
  });
});
```

Change the function name, the answers and the expected result to match your own rules. `npm test` runs every test once and reports which pass.

Choosing your five tests

Five tests should cover the outcomes and the edges of your rules, not five versions of the same case:

- at least one **eligible** case, one **not eligible** case and one **partially eligible** case;
- a **boundary**: an answer right at the edge of a rule, such as the top of an income band;

- your **trickiest rule**: the one the team argued about.

If you wrote your rules as a table of cases during planning (these answers give this result), each row is already a test.

Test first, and who decides "correct"

You can ask Claude to write tests, and log it. But write the expected results yourself. If Claude writes both the code and the tests, a mistake in its understanding appears in both, and the tests pass anyway. A strong pattern is **test first**: write a failing test for a rule, then ask Claude to write the code that makes it pass. You keep the judgement about what correct means; Claude does the typing. That is augmentation, not abdication.

End-to-end tests (stretch)

Playwright drives a real browser through the whole journey: start page, every question, check your answers, result. It can also press `Tab` through the pages and check nothing scrolls sideways at 320px, so your accessibility checks run every time you change something. It is a stretch goal; only a few teams usually get there.

Check your understanding

1. What makes a function pure, and why does that make it easy to test?
2. Why install `vitest@1` rather than the latest Vitest for this starter?
3. Why should you write the expected results of your tests yourself?

6. Working with AI as a Team

The AI log

In UK public sector delivery, teams using AI tools are expected to show that they reviewed and validated AI-generated code before it reached citizens, not just that they used AI. An AI log with only "used Claude to write the component" entries shows AI was used but not governed.

`AI_LOG.md` is already in your starter, with one example entry. Each entry has four fields:

Field	What to write
Date	When you did it
Task	What you asked the AI to help with
What AI Generated	What it produced, including anything wrong or missing
What You Changed + Why	What you changed before committing, and the reason

What an assessor looks for is the last field: evidence that you exercised independent judgement. You caught a hallucination, caught an accessibility miss, chose a different pattern, or added the error state the AI left out. Aim for three to five **different patterns** across your entries: planning, multi-agent critique, plan-mode scaffolding, refactoring and test generation. Write your first entry before your first line of code (planning counts), and your last before you present.

Where AI-generated code usually needs correcting

Teams building this service tend to find the same problems:

- it says a change is done, or correct, when it is not;
- it builds the summary as a `<table>`, or the error summary as a plain styled box;
- it writes generic error messages;
- it leaves out ARIA attributes and labels unless asked;
- it adds features nobody asked for;
- it chooses between your CSS and govuk-frontend instead of combining them, or ignores parts of the scaffold;
- asked to review its own work, it keeps finding more, so self-review never finishes.

Specific prompts get specific results. "Make it accessible" gets you something generic; "every radio group needs a `<fieldset>` with the question as its `<legend>`" gets you what you meant.

Ways of working that teams found useful

- **Plan first, then build in small steps:** plan, do one step, review it, then the next.

- **One owner per page**, with pull requests and a teammate reviewing each one, to avoid merge conflicts.
- **Parallel Claude sessions**, one per page or task, with each session writing to the AI log so the others can see what it did.
- **A larger model to plan and smaller sub-agents to implement** against a fixed plan, which keeps each one's context narrow.
- **Asking Claude for a gap analysis** against the acceptance criteria, then closing the gaps one at a time.

Models and usage limits

Claude Code may default to the largest, most expensive model. For most of this build, a mid-sized model is enough and uses your usage allowance more slowly. `/model` shows and changes the model. If you hit a usage limit, switching to a smaller model can give you more room; tell a facilitator if you are blocked.

Key idea. The process matters more than the product. How you planned, which files you gave Claude, and how you checked its work is what makes this a portfolio piece.

Check your understanding

1. What does an assessor look for in an AI log entry?
2. Name three things AI-generated code in this build often gets wrong.
3. Why is "make it accessible" a weak prompt, and what would be better?

7. Presenting and Reflecting

The presentation

Each team gives a five-minute live demo, followed by one or two questions. No slides are needed. Show, in this order:

1. an **eligible** user, end to end;
2. a **not eligible** path;
3. one **validation error**, pointing out the error summary and the inline message;
4. one **AI log entry**: what the AI did and what you changed;
5. your **tests passing**.

If you have time, justify one design decision against the GOV.UK **Service Standard**: for example how you held form state, why the summary is a `<dl>`, or how you kept the eligibility logic testable. Point 4 (make the service simple to use), point 5 (make sure everyone can use the service) and point 13 (use and contribute to open standards, common components and patterns) are the most relevant. Explain the reasoning, not just what you did, and name one thing you would do next.

Peer feedback

After the presentations, each team reviews two other teams' services against the rubric. For each row, write one specific strength and one specific improvement. Check in particular: does it meet each requirement, can it be used with the keyboard alone, and are the error messages clear? This is the kind of review a service assessment panel carries out.

Reflection

The retrospective and your own reflection turn two days of building into something you can use at work. Useful questions:

- Which AI tool or workflow helped most, and on what task? What would have been faster by hand?
- Where did AI generate something that looked right but broke a GOV.UK pattern? Re-read your AI log.
- What was your most significant architecture decision, and would you make it again?
- Which GOV.UK pattern was hardest, and what did it require that you would not have included on your own?
- Which accessibility problem did manual testing find that an automated tool would have missed?
- What would need to change before this prototype could be a real government service? Name three gaps.

Write three to five takeaways for your own work before you leave.

Check your understanding

1. What five things does the presentation show, and in what order?
2. Which Service Standard points are most relevant to this service?
3. Name one gap between your prototype and a production-ready government service.

References and Further Reading

The chapters draw on the Week 3 course materials (the brief, the plan of action, the lab README and starter) and the programme courseware. The sources below back specific points in the text and are the best places to go next. Links were checked in September 2026.

Chapters 1 and 3: GOV.UK patterns and the Service Standard

Service Standard (GOV.UK Service Manual)
https://www.gov.uk/service-manual/service-standard
The 14 points every government service is assessed against.

Start using a service (GOV.UK Design System)
https://design-system.service.gov.uk/patterns/start-using-a-service/
What a start page contains and where it lives.

Question pages (GOV.UK Design System)
https://design-system.service.gov.uk/patterns/question-pages/
One thing per page, back links, headings as labels, Continue buttons.

Check answers (GOV.UK Design System)
https://design-system.service.gov.uk/patterns/check-answers/
The check your answers page and Change links.

Summary list (GOV.UK Design System)
https://design-system.service.gov.uk/components/summary-list/
Why the summary is a `<dl>`, and visually hidden text on Change links.

Error summary (GOV.UK Design System)
https://design-system.service.gov.uk/components/error-summary/
Placement, focus, links to fields and the "Error:" title prefix.

Error message (GOV.UK Design System)
https://design-system.service.gov.uk/components/error-message/
How to write specific, useful error messages.

Confirmation pages (GOV.UK Design System)
https://design-system.service.gov.uk/patterns/confirmation-pages/
How to show the outcome and next steps.

GOV.UK Frontend
https://frontend.design-system.service.gov.uk/
The official production component library, for comparison with the teaching scaffold.

Chapter 2: Planning

Sharing state between components (React)

https://react.dev/learn/sharing-state-between-components
Lifting state up, the fix for answers lost between pages.

How Claude remembers your project (Claude Code docs)
https://code.claude.com/docs/en/memory
What `CLAUDE.md` is, where it lives and how to write instructions Claude follows.

Architectural Decision Records
https://adr.github.io/
Templates and examples for recording decisions.

Chapter 4: Accessibility

Understanding accessibility requirements for public sector bodies (GOV.UK)
https://www.gov.uk/guidance/accessibility-requirements-for-public-sector-websites-and-apps
The 2018 regulations, WCAG 2.2 AA and the accessibility statement.

Sample accessibility statement (GOV.UK)
https://www.gov.uk/government/publications/sample-accessibility-statement
A template for the statement your service needs.

What's new in WCAG 2.2 (W3C)
https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/
The nine new success criteria and their levels.

How to Meet WCAG (Quick Reference) (W3C)
https://www.w3.org/WAI/WCAG22/quickref/
Every success criterion, filterable to level AA.

Pa11y
https://pa11y.org/
The command-line accessibility checker.

Chapter 5: Testing

Getting started (Vitest)
https://vitest.dev/guide/
Installing Vitest, naming test files and running tests.

Accessibility testing (Playwright)
https://playwright.dev/docs/accessibility-testing
Adding automated accessibility scans to end-to-end tests.

Appendices

A. Setup and command cheat sheet

```
mkdir -p ~/Documents/my-work/w03-hackathon
cd ~/Documents/my-work/w03-hackathon
# copy the starter: see the lab README for the command
npm install
npm run dev
```

Command	What it does
`npm install`	Install the project's dependencies
`npm run dev`	Start the development server on port 5002
`hostname`	Print the lab machine's name, for the address `https://<hostname>:5002`
`VITE_PORT=5003 npm run dev`	Start the server on another port in the 5000–5005 range
`npm install -D vitest@1`	Add the Vitest test runner
`npm test`	Run your tests (after adding the test script)
`Ctrl+C`	Stop the development server
`claude`	Start Claude Code in the current folder
`Shift+Tab`	Cycle Claude Code permission modes, including plan mode
`/model`	See or change Claude's model
`/init`	Draft a `CLAUDE.md` for the project

B. Glossary

Term	Meaning
Accessibility statement	A page public sector services must publish, saying how accessible the service is and how to report problems.
ADR	Architectural Decision Record: a short dated note of a decision, the options considered and why one was chosen.
AI log	Your team's record of AI-assisted work: task, what the AI generated, what you changed and why.
Contrast ratio	How different a text colour is from its background. Normal text needs at least 4.5:1.

Term	Meaning
Error summary	A box at the top of a page listing every error, each linking to its field.
GOV.UK Design System	The official styles, components and patterns for UK government services.
Lifting state up	Moving shared data to the component that contains all the components that use it.
MVS	Minimum Viable Submission: the six items every team must ship first.
One thing per page	The GOV.UK pattern of asking one question on each page.
Phase banner	A strip under the header saying the service is in alpha or beta, with a feedback link.
Pure function	A function that only uses what it is given and only returns a result.
RAID log	A record of risks, actions, issues and decisions.
Reflow	Content rearranging to fit a narrow screen without sideways scrolling.
Service Standard	The 14 points government services are assessed against.
State	Data a React app remembers while it runs, such as the user's answers.
Summary list	The GOV.UK component for key-value pairs, built as a `<dl>`.
Unit test	A small piece of code that checks one function gives the expected result.
WCAG 2.2 AA	The accessibility standard UK public sector services must meet.

C. Templates

AI log entry

```
## Instance N: <short title>

| Field | Detail |
|-------|--------|
| **Date** | YYYY-MM-DD |
| **Task** | What you asked the AI to help with |
| **What AI Generated** | What it produced, including what was wrong or missing |
| **What You Changed + Why** | What you changed before committing, and why |
```

Eligibility rules

Question	Answers you offer	Effect on eligibility
Property type		
Ownership		
Household income band		
Existing insulation		
Current heating		

Decision record

```
# Decision: <title>
Date: YYYY-MM-DD

## Context
What needed deciding, and why.

## Options
- Option A
- Option B

## Decision
What we chose.

## Consequences
What this makes easier or harder.
```

D. Accessibility walkthrough checklist

- **Keyboard:** every element reachable with Tab, focus always visible, nothing traps focus, whole journey completed.
- **Labels:** every input has a label; clicking the label selects the option.
- **Contrast:** normal text at least 4.5:1.
- **Reflow:** no sideways scrolling at 320px or 400% zoom; nothing cut off or overlapping.
- **Screen reader:** one field's label and error message are announced.
- **Automated:** Pa11y or axe reports no errors, and you have still done the checks above.

E. Suggested answers to Check your understanding

Chapter 1

1. Multi-page question flow; check your answers; confirmation page; GOV.UK components; WCAG 2.2 AA; AI usage log.

2. The final submission.
3. To direct the AI, review what it produces and decide what ships.

Chapter 2

1. Each page's state is lost when the page unmounts on navigation. Hold all answers in one state object in `App` and pass values and setters down.
2. So the rules are in one place, separate from the pages, and can be tested without rendering anything.
3. Standing instructions for the project, such as the design system and conventions. Claude Code reads it at the start of every session.

Chapter 3

1. Key-value pairs are not tabular data; a table implies relationships the rows do not have, and screen readers announce it wrongly.
2. `role="alert"`, the heading "There is a problem", and a link from each error to its field. Focus should also move to it.
3. For example: "Select how your home is currently heated".
4. Phase banner, footer, accessibility statement and cookie notice.

Chapter 4

1. The Public Sector Bodies (Websites and Mobile Applications) (No. 2) Accessibility Regulations 2018. They must publish an accessibility statement.
2. At least 4.5:1. Inspect the text in browser DevTools and read the ratio in the colour picker, or use a contrast checker.
3. Automated tools find only some problems; many, such as a confusing focus order or an unusable journey, need a person to find them.

Chapter 5

1. It only uses its inputs and only returns a result, so a test can call it with a state object and check the result without running the app.
2. The latest Vitest needs a newer Vite than the starter uses; version 1 works with the starter's Vite 5.
3. If Claude writes both the code and the tests, a misunderstanding appears in both and the tests pass anyway. You decide what correct means.

Chapter 6

1. Evidence of independent judgement: what was wrong or missing in the AI's output and what you changed, and why.
2. For example: a table instead of a `<dl>`; generic error messages; missing ARIA and labels; unrequested features; claiming work is correct when it is not.
3. It is vague, so the output is generic. Name the specific requirement, for example a `<fieldset>` with the question as its `<legend>`.

Chapter 7

1. An eligible path; a not eligible path; a validation error; an AI log entry; passing tests.
2. Point 4 (simple to use), point 5 (everyone can use the service) and point 13 (open standards, common components and patterns).
3. For example: a real rules source from policy, a backend and data storage, security review, user research with real citizens, or a full accessibility audit.